

Foundations for Trustworthy Parallel Computing

Duration: 3 Hours

Maximum Marks: 40

Instructions:

- Assume Sequential Consistency (SC) unless otherwise specified.
- Assume standard Hoare Logic axioms and inference rules unless stated otherwise.

Important: Please ensure your answers are clearly written and well-structured. Partial credit will be given for correct reasoning even if the final answer is incomplete.

Q1. Threads and Abstract Parallelism [2 marks]

Let T_i and T_j be two different threads of a concurrent program S . Assuming concurrency as abstract parallelism, which of the following statements are *always true*?

- A. T_i and T_j can share CPU. $\top$
- B. Execution of T_i cannot influence T_j . F
- C. T_i and T_j represent potential parallelism. $\top$

MCA [+1 for correct
0 if wrong selected]

Q2. Mathematical Induction Basics [2 marks]

Consider the following statements where N is the set of all natural numbers and P is a property of natural numbers (i.e., $P : N \rightarrow Prop$).

- S1. $P(0)$
- S2. $P(k) \rightarrow P(k+1)$
- S3. $P(n)$
- S4. $\forall n \in N, P(n)$

Which of the following options are correct?

- A. Principle of mathematical induction helps us prove S3 by proving S1 and S2.
- B. Principle of mathematical induction helps us prove $(\forall n \in N, S3)$ by proving S1 and S2.
- ☒ C. Principle of mathematical induction helps us prove S4 by proving S1 and $(\forall k \in N, S2)$.
- D. None of the above.

Q3. Hoare Logic Inference Rules

[3 marks]

Consider the following statements where $\{P\}S\{Q\}$ represents the Hoare triple used for specifying the partial correctness of program S .

- A. If $\{P\}S\{Q\}$ and $Q \rightarrow R$ then $\{P\}S\{R\}$ ✓
- B. If $\{P\}S\{Q\}$ and $R \rightarrow P$ then $\{P\}S\{R\}$ ✗
- C. If $\{P\}S\{Q\}$ and $P \rightarrow R$ then $\{R\}S\{Q\}$ ✗
- D. If $\{P\}S\{Q\}$ and $R \rightarrow P$ then $\{R\}S\{Q\}$ ✓
- E. If $\{P\}S\{Q\}$ and $R \rightarrow Q$ then $\{P\}S\{R\}$ ✗
- F. If $\{P\}S_1\{Q_1\}$, $Q_1 \rightarrow Q_2$, and $\{Q_2\}S_2\{Q\}$ then $\{P\}S\{Q\}$ ✓

$S = S_1; S_2$

mentioned in class

Which of the above statements are correct?

Q4. Swap Program Verification

[3 marks]

Let S be the following sequential program to swap the values in the variables x and y .

$t := x;$ $\rightarrow \{x = x, y = y\} \rightarrow +1$
 $x := y;$ $\rightarrow \{y = y, t = x\}$
 $y := t;$ $\rightarrow \{x = y, y = x\} \rightarrow +1$
 $\rightarrow \{x = y, t = x\} \rightarrow +1$

Write the partial correctness specification for S using a Hoare triple and justify its correctness using the Axioms and Inference Rules for Hoare logic.

Q5. Factorial Program Verification

[4 marks]

Let S be the following factorial program:

$y := 1;$
while ($x \neq 0$) {
 $y := y * x;$
 $x := x - 1;$
}

Invariant $\rightarrow \boxed{x! \cdot y = x!}$

Write a partial correctness specification for S using a Hoare triple and justify its correctness using the Axioms and Inference Rules for Hoare logic. Your solution must include:
(1) a suitable loop invariant, (2) proof of invariant initialization, (3) proof of invariant preservation, and (4) proof that invariant + loop condition implies the postcondition.

Q6. Execution Graphs and SC

[6 marks]

(a) What is an execution graph? Explain its use in the context of concurrent programs by describing its various components: nodes, *po*, *rf*, and *mo* relations. (3 marks)

(b) Describe Sequential Consistency using a declarative specification on execution graphs. (3 marks)

Q7. Happens-Before and WMM

[8 marks]

(a) Define the happens-before (*hb*) relation and explain how it can be used to give a declarative specification for the Weak Release/Acquire (WRA) memory model. (3 marks)

1 (hb)
1 (some algo)
1 (exact paper)
WRA (definition)

(b) Compare the Release/Acquire (RA) model and the Strong Release/Acquire (SRA) model by describing the respective consistency constraints on execution graphs. (3 marks)

→ (hb vmo) acycle

(c) Provide simple litmus test programs that differentiate WRA from RA, and RA from SRA memory models. (2 marks)

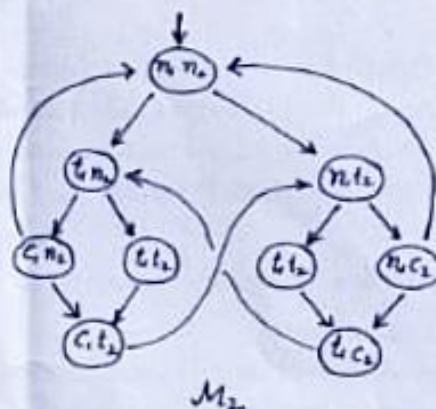
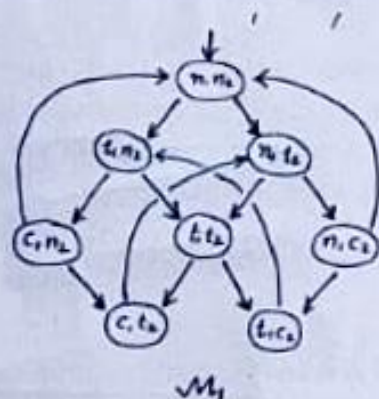
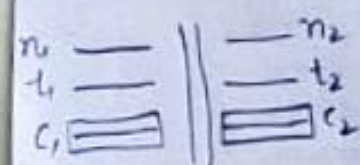
WW

2+2W

Q8. Deadlock and Starvation Analysis

[6 marks]

Consider two processes, each of which is in its non-critical section (*n*), or trying to enter its critical section (*t*), or in its critical section (*c*). Each individual process undergoes transitions in the cycle $n \rightarrow t \rightarrow c \rightarrow n \rightarrow \dots$, but the two processes interleave with each other. The goal is to design a protocol for determining which process is allowed to enter its critical section at which time, in order to ensure mutual exclusion of critical sections.



3 → Both Deadlock prevention
1.5 → M1 → starvation
1.5 → M2 → no

No starvation!

Figure 1: Transition systems M_1 (ALG1) and M_2 (ALG2)

Assume that we came up with two different protocols (ALG1 and ALG2) for solving mutual exclusion of critical sections. Also given are the following transition systems M_1 and M_2 corresponding to ALG1 and ALG2 respectively. (Note: Refer to the image for the state machine diagrams of M_1 and M_2 , where different states may have same label)

Discuss the effectiveness of ALG1 and ALG2 while evaluating them with respect to Dead-

Q9. Peterson's Algorithm Analysis

[6 marks]

Consider Peterson's solution for the two-process critical section problem:

```
turn := 1; wantp := false; wantq := false;
```

Process P:

```
p1: //Non-Critical Section
p2: wantp := true
p3: await (turn = 1  $\vee$  wantq = false)
p4: Critical Section
p5: wantp := false
p6: turn := 2
```

Process Q:

```
q1: //Non-Critical Section
q2: wantq := true
q3: await (turn = 2  $\vee$  wantp = false)
q4: Critical Section
q5: wantq := false
q6: turn := 1
```

Now consider the following statements:

 $S_1: p_1 \dots p_5 \leftrightarrow \text{wantp} \quad \top$ $S_2: \neg(p_4 \wedge q_4)$ $S_3: \text{turn} = 1 \vee \text{turn} = 2$ $S_4: (p_2 \wedge q_2) \rightarrow \Diamond(p_4 \vee q_4)$ $S_5: p_2 \rightarrow \Diamond p_4$ $S_6: q_2 \rightarrow \Diamond q_4$

Temporal Operator: The symbol $\Diamond$ (diamond) means "eventually" in Linear Temporal Logic (LTL). For any property ϕ , the formula $\Diamond\phi$ is true along an execution if ϕ holds at some future point in that execution (including possibly immediately). For example, $p_2 \rightarrow \Diamond p_4$ means: if Process P reaches p_2 , then eventually it will reach p_4 .

Which of the following statements are correct?

- not considered
- ⁺² ☒ A. S_1 is an invariant
 - ⁺² ☒ B. S_2 represents mutual exclusion of critical section and its proof requires S_3
 - ⁺² ☒ C. S_4 represents freedom from deadlock
 - ⁺² ☒ D. S_5 and S_6 combinely represent freedom from starvation for P and Q
- freedom from starvation

Justify your answer by selecting all correct options and explaining your reasoning.